

Assignment 04 Solutions: OOP & Operator Overloading

Student Name: ahmed mohamed saed nofal

Course Code: ONL5_SWD5_S4

Group Name: ONL5_SWD5_S4

Topic: Point3D Class, Maths Class, & Duration Class

First Project: 3D Point Class

1 & 2 & 5 & 6. Point3D Class Implementation

```
using System;

public class Point3D : IComparable, ICloneable
{
    public int X { get; set; }
    public int Y { get; set; }
    public int Z { get; set; }

    public Point3D() : this(0, 0, 0) { }
    public Point3D(int x) : this(x, 0, 0) { }
    public Point3D(int x, int y) : this(x, y, 0) { }
    public Point3D(int x, int y, int z)
    {
        X = x; Y = y; Z = z;
    }

    public override string ToString()
    {
        return $"Point Coordinates: ({X}, {Y}, {Z})";
    }

    public static bool operator ==(Point3D p1, Point3D p2)
    {
        if (ReferenceEquals(p1, p2)) return true;
        if (p1 is null || p2 is null) return false;
        return p1.X == p2.X && p1.Y == p2.Y && p1.Z == p2.Z;
    }

    public static bool operator !=(Point3D p1, Point3D p2) => !(p1 == p2);

    public override bool Equals(object obj) => obj is Point3D other && this == other;

    public override int GetHashCode() => GetHashCode.Combine(X, Y, Z);

    public int CompareTo(object obj)
    {
        if (obj is Point3D other)
        {
            if (this.X != other.X) return this.X.CompareTo(other.X);
            return this.Y.CompareTo(other.Y);
        }
        return 1;
    }

    public object Clone() => new Point3D(this.X, this.Y, this.Z);
}
```

Second Project: Maths Class

```
public static class Maths
{
    public static double Add(double a, double b) => a + b;
    public static double Subtract(double a, double b) => a - b;
    public static double Multiply(double a, double b) => a * b;
    public static double Divide(double a, double b) => b == 0 ? 0 : a / b;
}
```

Third Project: Duration Class & Operator Overloading

```
using System;

public class Duration
{
    public int Hours { get; set; }
    public int Minutes { get; set; }
    public int Seconds { get; set; }

    public Duration(int hours, int minutes, int seconds)
    {
        Hours = hours; Minutes = minutes; Seconds = seconds;
        Normalize();
    }

    public Duration(int totalSeconds)
    {
        Hours = totalSeconds / 3600;
        Minutes = (totalSeconds % 3600) / 60;
        Seconds = totalSeconds % 60;
    }

    private void Normalize()
    {
        int totalSec = ToTotalSeconds();
        if (totalSec < 0) totalSec = 0;
        Hours = totalSec / 3600;
        Minutes = (totalSec % 3600) / 60;
        Seconds = totalSec % 60;
    }

    public int ToTotalSeconds() => (Hours * 3600) + (Minutes * 60) + Seconds;

    public override string ToString()
    {
        if (Hours > 0) return $"Hours: {Hours}, Minutes :{Minutes}, Seconds :{Seconds}";
        if (Minutes > 0) return $"Minutes :{Minutes}, Seconds :{Seconds}";
        return $"Seconds :{Seconds}";
    }

    public override bool Equals(object obj) => obj is Duration other && this.ToTotalSeconds() == other.ToTotalSeconds();
    public override int GetHashCode() => ToTotalSeconds().GetHashCode();

    public static Duration operator +(Duration d1, Duration d2) => new
    Duration(d1.ToTotalSeconds() + d2.ToTotalSeconds());
    public static Duration operator +(Duration d1, int seconds) => new
```

```

Duration(d1.ToTotalSeconds() + seconds);
    public static Duration operator +(int seconds, Duration d1) => d1 + seconds;
    public static Duration operator ++(Duration d) => new Duration(d.ToTotalSeconds() + 60);
    public static Duration operator --(Duration d) => new Duration(d.ToTotalSeconds() - 60);
    public static Duration operator -(Duration d1, Duration d2) => new
Duration(d1.ToTotalSeconds() - d2.ToTotalSeconds());

    public static bool operator >(Duration d1, Duration d2) => d1.ToTotalSeconds() >
d2.ToTotalSeconds();
    public static bool operator <(Duration d1, Duration d2) => d1.ToTotalSeconds() <
d2.ToTotalSeconds();
    public static bool operator <=(Duration d1, Duration d2) => d1.ToTotalSeconds() <=
d2.ToTotalSeconds();
    public static bool operator >=(Duration d1, Duration d2) => d1.ToTotalSeconds() >=
d2.ToTotalSeconds();

    public static bool operator true(Duration d) => d != null && d.ToTotalSeconds() > 0;
    public static bool operator false(Duration d) => d == null || d.ToTotalSeconds() <= 0;

    public static explicit operator DateTime(Duration d) => new DateTime(1, 1, 1, d.Hours % 24,
d.Minutes, d.Seconds);
}

```